第11章

面向对象的架构（OOA）

陈龙彪
longbiaochen@xmu.edu.cn

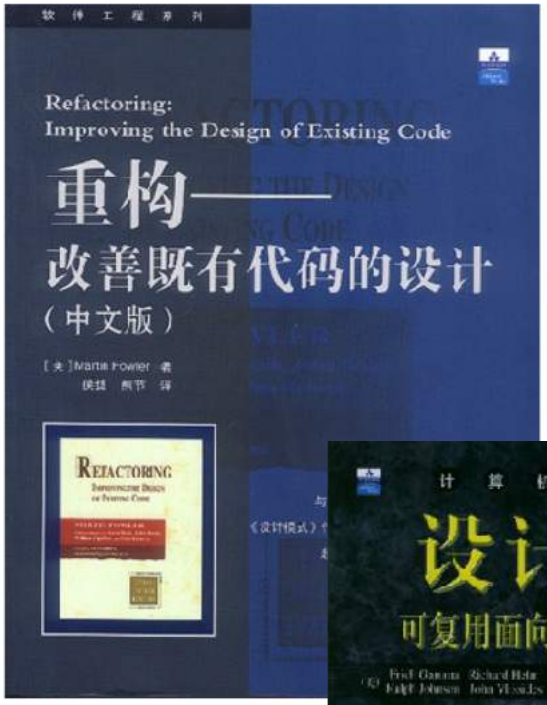
厦门大学信息学院
2019年4月

第一部分

面向对象设计原则

本章教学内容

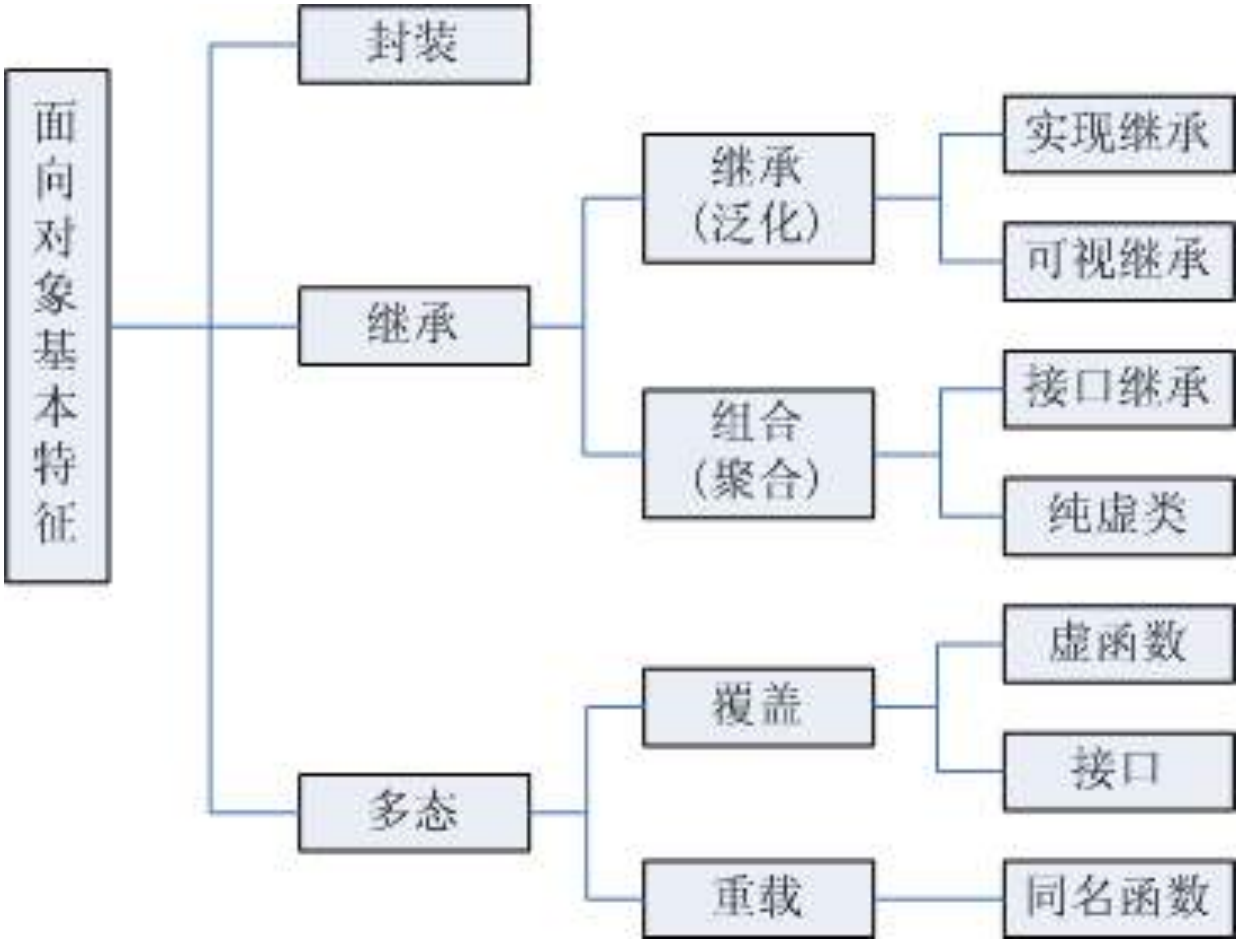
- 面向对象设计原则概述
- 单一职责原则
- 开闭原则
- 里氏代换原则
- 依赖倒转原则
- 接口隔离原则
- 合成复用原则
- 迪米特法则





面向对象

- 复习课

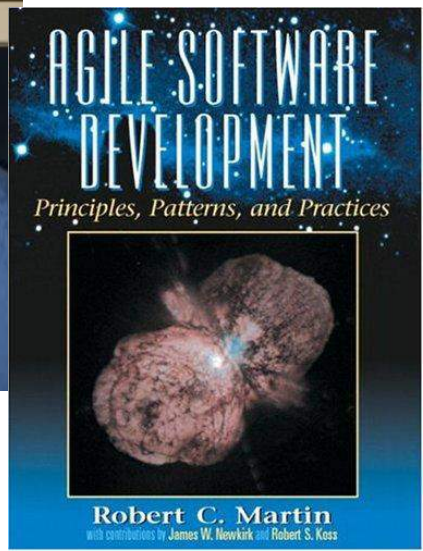

面向对象基本特征
封装
继承
继承(泛化)
实现继承
可视继承
组合(聚合)
接口继承
纯虚类
多态
覆盖
虚函数
接口
重载
同名函数

面向对象设计原则概述

- 软件的可维护性和可复用性
  - Robert C. Martin认为一个可维护性（Maintainability）较低的软件设计，通常由于如下4个原因造成：
    - 过于僵硬(Rigidity)
    - 过于脆弱(Fragility)
    - 复用率低(Immobility)
    - 黏度过高(Viscosity)



Robert C. Martin



Uncle Bob

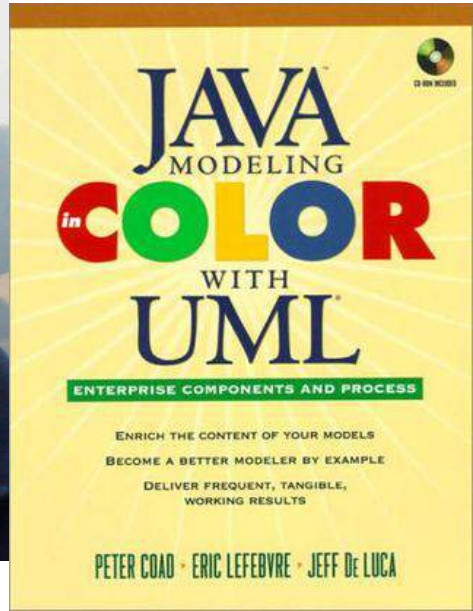
面向对象设计原则概述

- 软件的可维护性和可复用性
  - Peter Coad认为一个好的系统设计应具备如下三个性质：
    - 可扩展性(Extensibility)
    - 灵活性(Flexibility)
    - 可插入性(Pluggability)

软件工程和建模大师

Peter Coad





面向对象设计原则概述

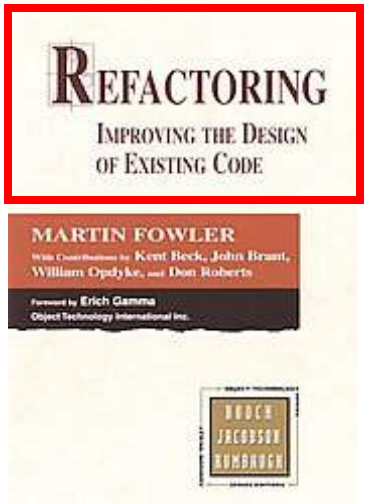
- 软件的可维护性和可复用性
  - 软件的复用（Reuse）拥有众多优点，如可以提高软件的开发效率，提高软件质量，节约开发成本，恰当的复用还可以改善系统的可维护性
  - 面向对象设计复用的目标在于实现可维护的复用
  - 在面向对象设计里，可维护性复用都是以面向对象设计原则为基础的，这些设计原则首先都是复用的原则
  - 遵循这些设计原则可以有效地提高系统的复用性，同时提高系统的可维护性

面向对象设计原则概述

- 软件的可维护性和可复用性
  - 面向对象设计原则和设计模式也是对系统进行合理重构的指南针，重构(Refactoring)是在不改变软件现有功能的基础上，通过调整程序代码改善软件的质量、性能，使其程序的设计模式和架构更趋合理，提高软件的扩展性和维护性。

Martin Fowler





面向对象设计原则概述

面向对象设计原则简介

- 常用的面向对象设计原则包括7个，这些原则并不是孤立存在的，它们相互依赖，相互补充。

设计原则名称	设计原则简介	重要性
单一职责原则 (Single Responsibility Principle, SRP)	类的职责要单一，不能将太多的职责放在一个类中	★★★★☆
开闭原则 (Open-Closed Principle, OCP)	软件实体对扩展是开放的，但对修改是关闭的，即在不修改一个软件实体的基础上去扩展其功能	★★★★★
里氏代换原则 (Liskov Substitution Principle, LSP)	在软件系统中，一个可以接受基类对象的地方必然可以接受一个子类对象	★★★★☆
依赖倒转原则 (Dependency Inversion Principle, DIP)	要针对抽象层编程，而不要针对具体类编程	★★★★★
接口隔离原则 (Interface Segregation Principle, ISP)	使用多个专门的接口来取代一个统一的接口	★★☆☆☆
合成复用原则 (Composite Reuse Principle, CRP)	在系统中应该尽量多使用组合和聚合关联关系，尽量少使用甚至不使用继承关系	★★★★☆
迪米特法则 (Law of Demeter, LoD)	一个软件实体对其他实体的引用越少越好，或者说如果两个类不必彼此直接通信，那么这两个类就不应当发生直接的相互作用，而是通过引入一个第三者发生间接交互	★★★☆☆

单一职责原则

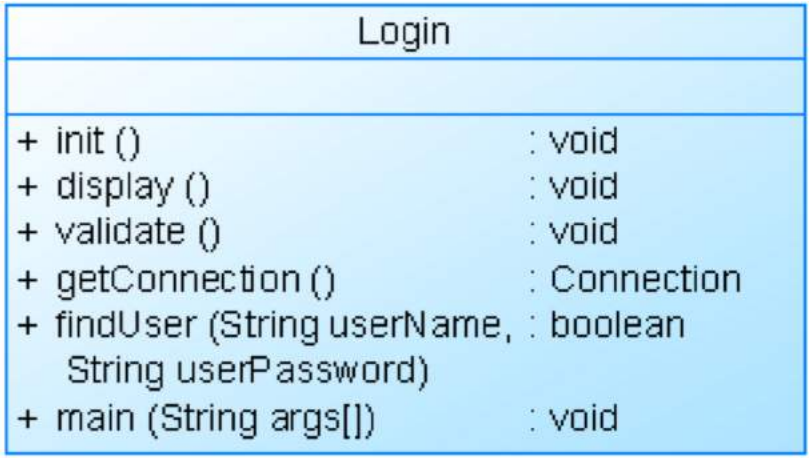
- 单一职责原则定义
  - 单一职责原则（Single Responsibility Principle, SRP）
    - 一个对象应该只包含单一的职责，并且该职责被完整地封装在一个类中。
  - 另一种定义方式如下：
    - 就一个类而言，应该仅有一个引起它变化的原因。

单一职责原则

- 单一职责原则分析
  - 一个类承担的职责越多，它被复用的可能性越小
  - 类的职责主要包括数据职责和行为职责：数据职责通过其属性来体现，而行为职责通过其方法来体现
  - 而且如果一个类承担的职责过多，相当于将这些职责耦合在一起，当其中一个职责变化时，可能会影响其他职责的运作
  - 单一职责原则是实现高内聚、低耦合的指导方针，它是最简单但又最难运用的原则，需要设计人员发现类的不同职责并将其分离

单一职责原则

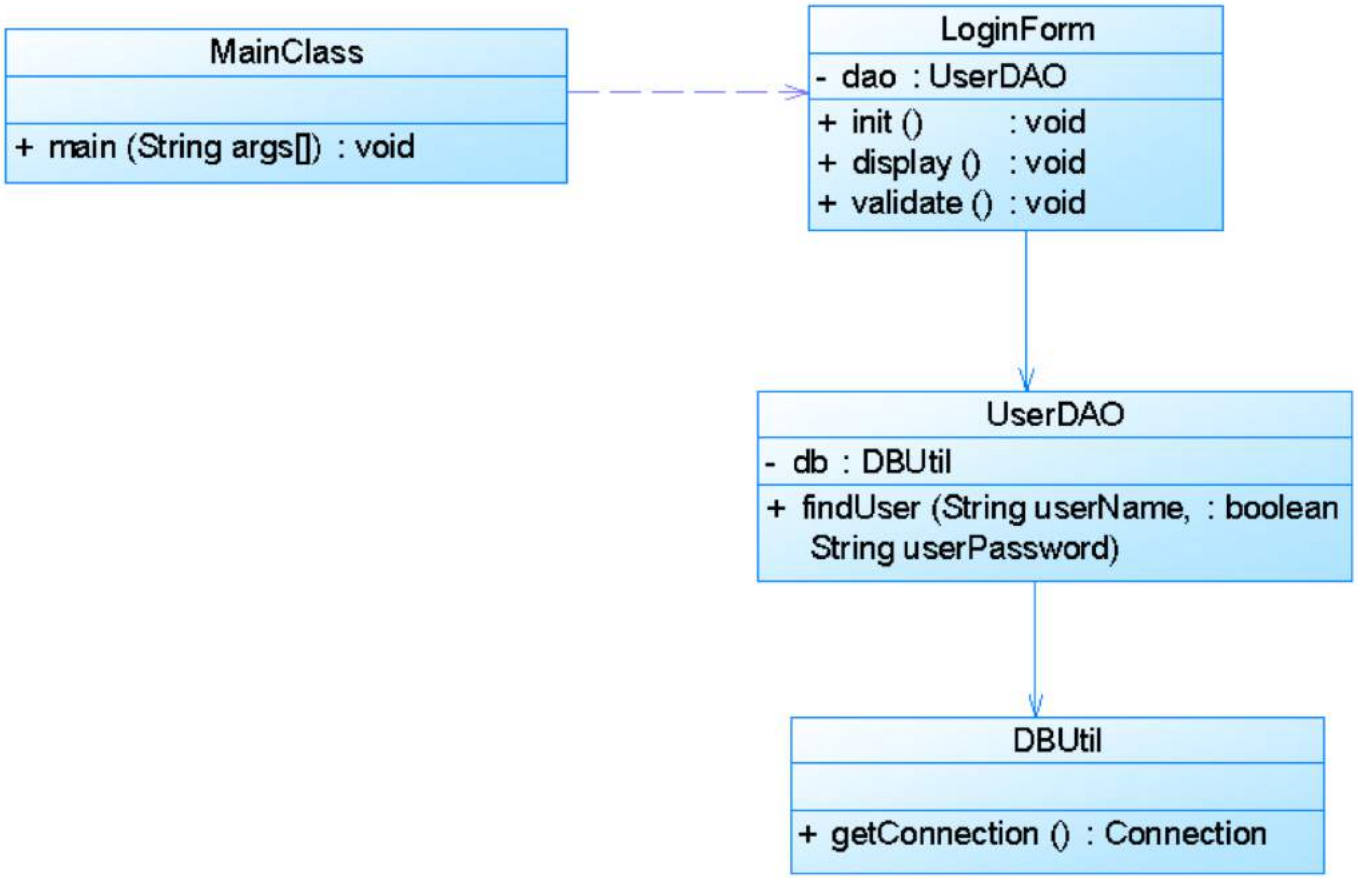
- 单一职责原则实例
  - 实例说明
    - 某基于Java的C/S系统的 “登录功能” 通过如下登录类实现：



    - 现使用单一职责原则对其进行重构。

单一职责原则

- 单一职责原则实例
  - 实例解析



开闭原则

- 开闭原则定义
  - 开闭原则（Open-Closed Principle, OCP）：一个软件实体应当对扩展开放，对修改关闭
  - 在设计一个模块的时候，应当使这个模块可以在不被修改的前提下被扩展，即实现在不修改源代码的情况下改变这个模块的行为

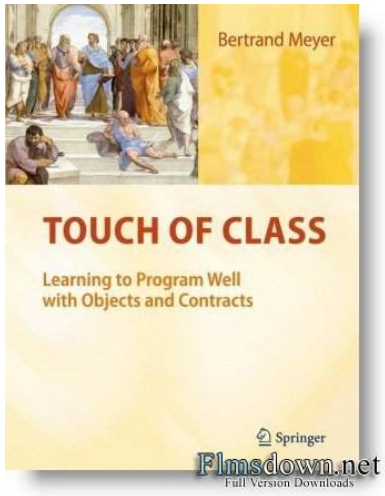
开闭原则

- 开闭原则分析
  - 开闭原则由Bertrand Meyer于1988年提出，它是面向对象设计中最重要的原则之一
  - 在开闭原则的定义中，软件实体可以指一个软件模块、一个由多个类组成的局部结构或一个独立的类



Bertrand Meyer



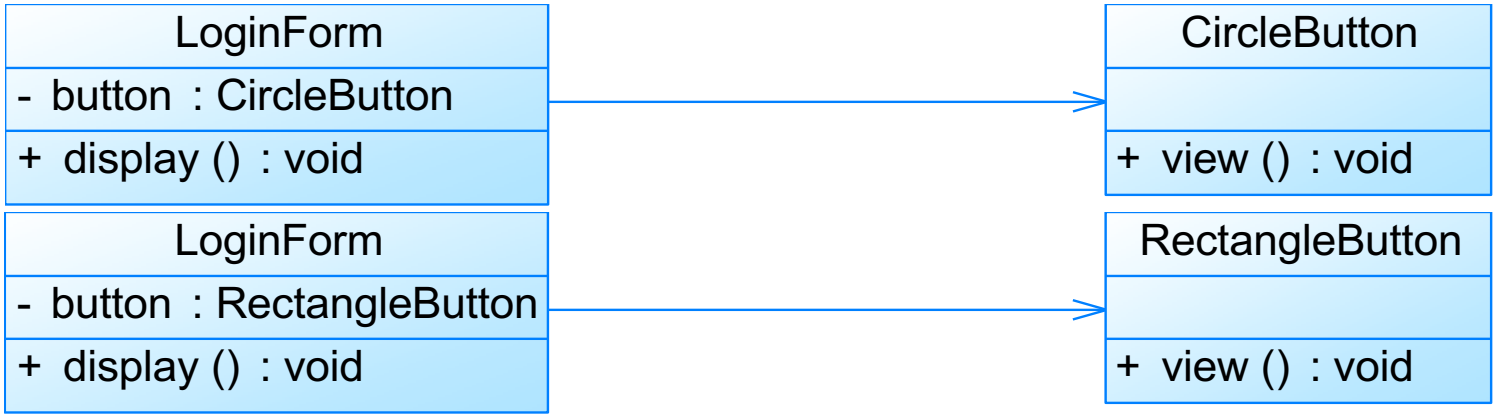


开闭原则

- 开闭原则分析
  - 抽象化是开闭原则的关键
  - 开闭原则还可以通过一个更加具体的对可变性封装原则来描述，对可变性封装原则（Principle of Encapsulation of Variation, EVP）要求找到系统的可变因素并将其封装起来

开闭原则

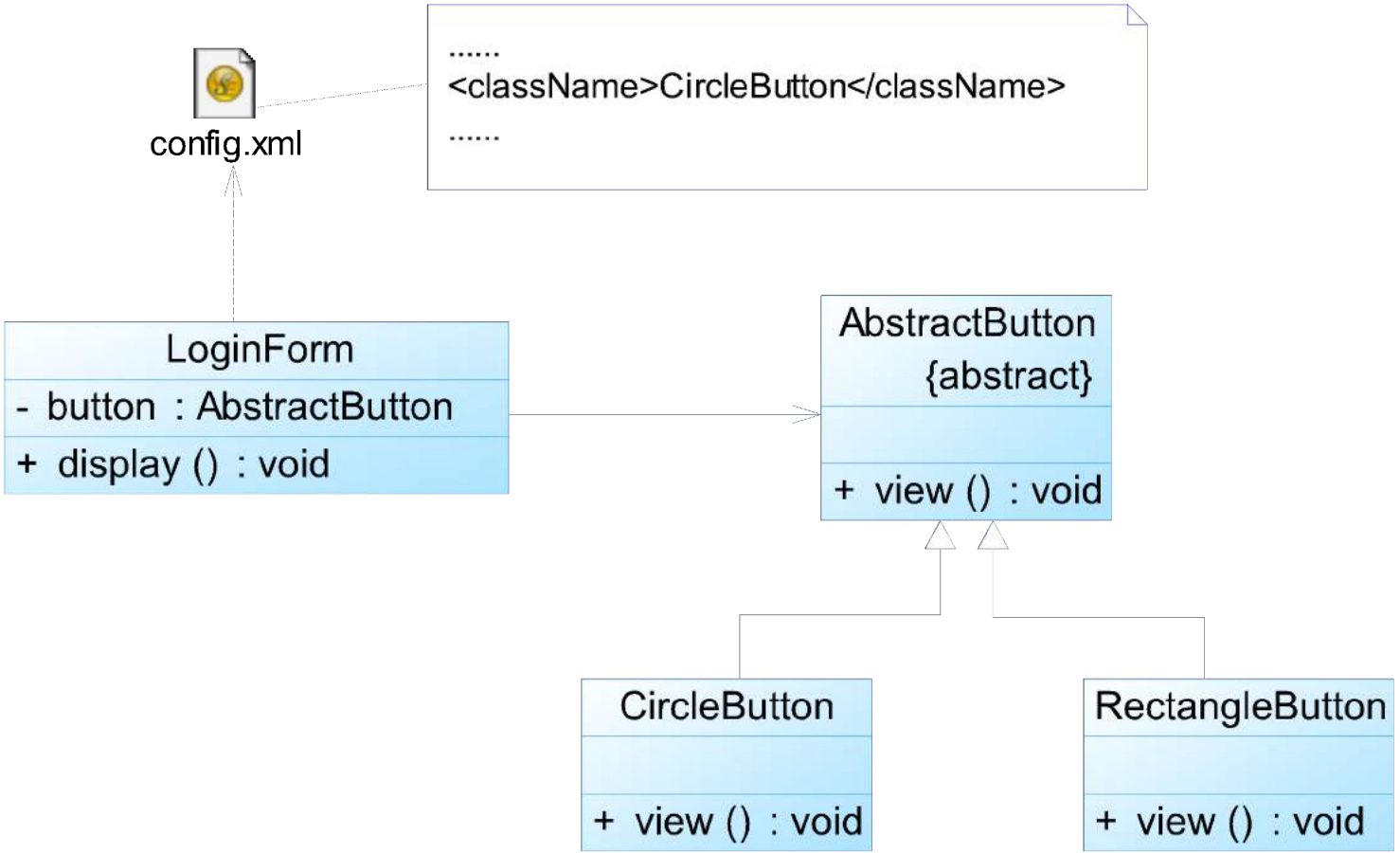
- 开闭原则实例
  - 实例说明
    - 某图形界面系统提供了各种不同形状的按钮，客户端代码可针对这些按钮进行编程，用户可能会改变需求要求使用不同的按钮，原始设计方案如图所示：



    - 现对该系统进行重构，使之满足开闭原则的要求。

开闭原则

- 开闭原则实例
  - 实例解析



里氏代换原则

- 里氏代换原则定义
  - 里氏代换原则（Liskov Substitution Principle, LSP）有两种定义方式
    - 如果对每一个类型为S的对象o1，都有类型为T的对象o2，使得以T定义的所有程序P在所有的对象o1都代换o2时，程序P的行为没有变化，那么类型S是类型T的子类型
    - 所有引用基类（父类）的地方必须能透明地使用其子类的对象

里氏代换原则

- 里氏代换原则分析
  - 里氏代换原则由2008年图灵奖得主、美国第一位计算机科学女博士、麻省理工学院教授Barbara Liskov和卡内基.梅隆大学Jeannette Wing教授于1994年提出。其原文如下：Let q(x) be a property provable about objects x of type T. Then q(y) should be true for objects y of type S where S is a subtype of T.



芭芭拉·利斯科夫（Barbara Liskov），美国计算机科学家，2008年图灵奖得主，2004年约翰.冯诺依曼奖得主，美国工程院院士，美国艺术与科学院院士，美国计算机协会会士。现任麻省理工学院电子电气与计算机科学系教授。她是美国第一个计算机科学女博士。

里氏代换原则

- 里氏代换原则分析
  - 里氏代换原则可以通俗表述为：在软件中如果能够使用基类对象，那么一定能够使用其子类对象
  - 把基类都替换成它的子类，程序将不会产生任何错误和异常，反过来则不成立
  - 里氏代换原则是实现开闭原则的重要方式之一，由于使用基类对象的地方都可以使用子类对象，因此在程序中尽量使用基类类型来对对象进行定义，而在运行时再确定其子类类型，用子类对象来替换父类对象

```
Horse h;
h = new WhiteHorse();
```

里氏代换原则

- 里氏代换原则分析

小明喜欢动物→小明喜欢猫

因为猫是动物 ☺





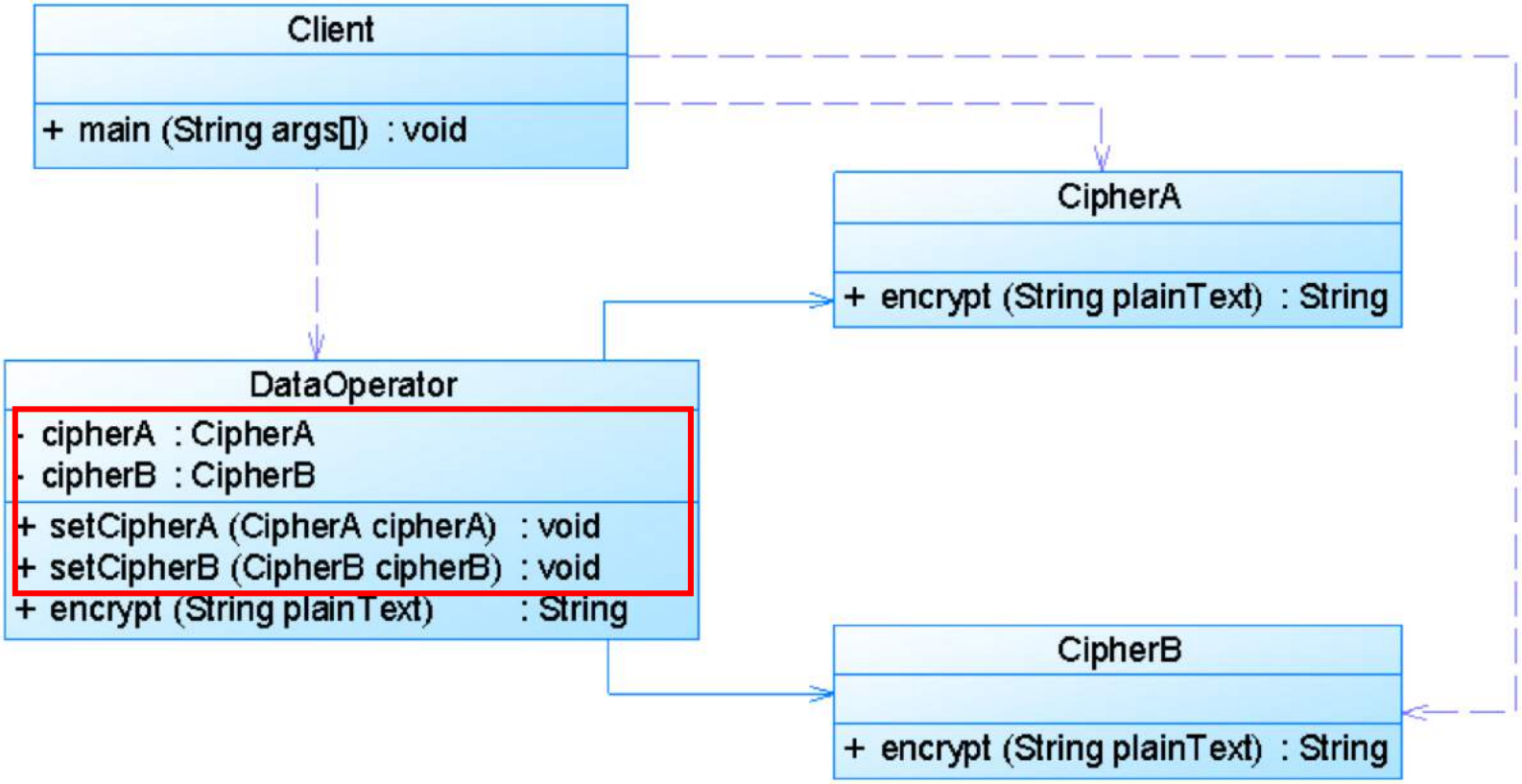


里氏代换原则

- 里氏代换原则实例
  - 实例说明
    - 某系统需要实现对重要数据（如用户密码）的加密处理，在数据操作类(DataOperator)中需要调用加密类中定义的加密算法，系统提供了两个不同的加密类，CipherA和CipherB，它们实现不同的加密方法，在DataOperator中可以选择其中的一个实现加密操作。
    - 如果需要更换一个加密算法类或者增加并使用一个新的加密算法类，如将CipherA改为CipherB，则需要修改客户类Client和数据操作类DataOperator的源代码，违背了开闭原则。
    - 现使用里氏代换原则对其进行重构，使得系统可以灵活扩展，符合开闭原则。

里氏代换原则

- 里氏代换原则实例
  - 实例说明



里氏代换原则

- 里氏代换原则实例
  - 实例解析



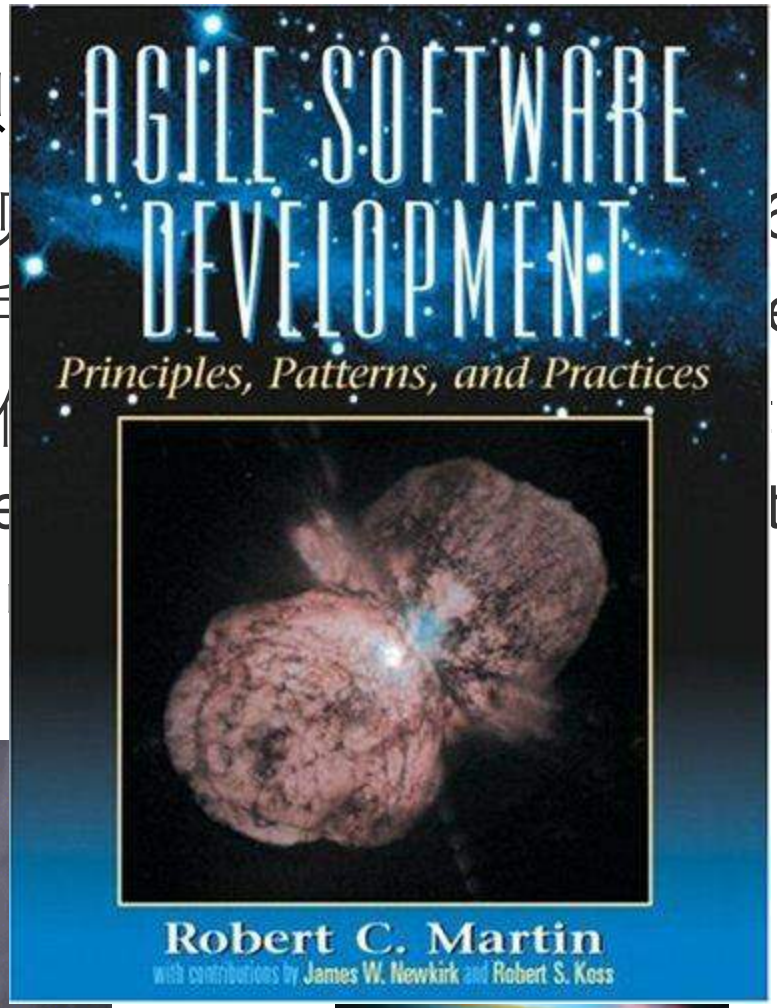
依赖倒转原则

- 依赖倒转原则定义
  - 依赖倒转原则（Dependence Inversion Principle, DIP）
    - 高层模块不应该依赖低层模块，它们都应该依赖抽象
    - 抽象不应该依赖于细节，细节应该依赖于抽象
    - 要针对接口编程，不要针对实现编程

Program to an interface, not an implementation.

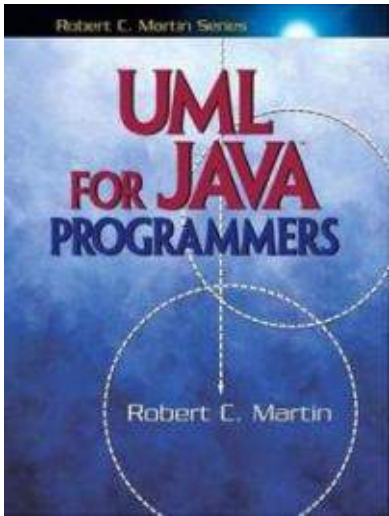
依赖倒转原则

- 依赖倒转原则
  - 依赖倒转原则 6年为《C++ Reporter》月 ebook的第三篇
  - 后来加入到 《Agile Software De terns, and Practices》









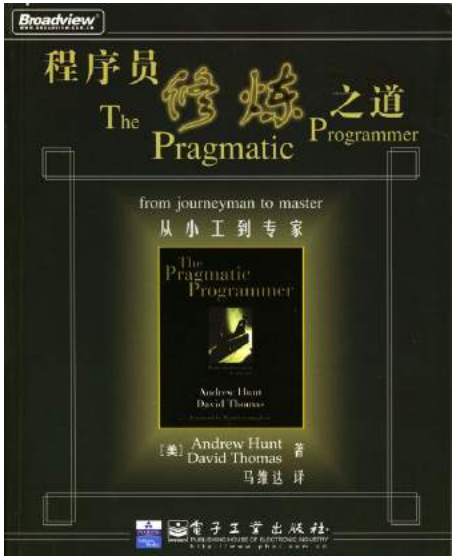
依赖倒转原则

- 依赖倒转原则分析
  - 代码要依赖于抽象的类，而不要依赖于具体的类；要针对接口或抽象类编程，而不是针对具体类编程
  - 实现开闭原则的关键是抽象化，并且从抽象化导出具体化实现，如果说开闭原则是面向对象设计的目标的话，那么依赖倒转原则就是面向对象设计的主要手段

依赖倒转原则

- 依赖倒转原则分析
  - 常用实现方式：代码中使用抽象类，而将具体类放在配置文件中
    - 将抽象放进代码，将细节放进元数据
    - Put Abstractions in Code, Details in Metadata

《程序员修炼之道：从小工到专家》(The Pragmatic programmer: from journeyman to master)

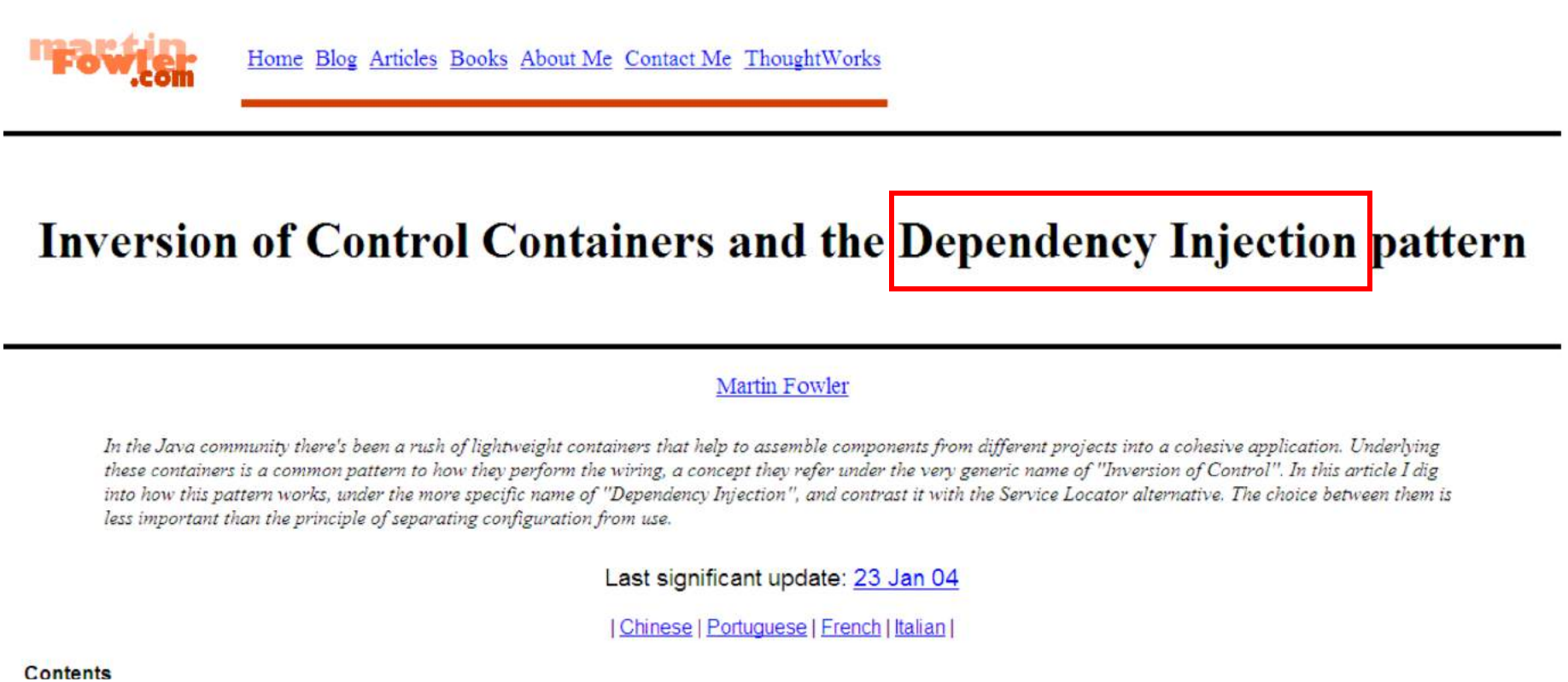


依赖倒转原则

- 依赖倒转原则分析
  - 类之间的耦合
    - 零耦合关系
    - 具体耦合关系
    - 抽象耦合关系
  - 依赖倒转原则要求客户端依赖于抽象耦合
  - 以抽象方式耦合是依赖倒转原则的关键

依赖倒转原则

- 依赖倒转原则分析
  - 依赖注入



martin Fowler .com

Home Blog Articles Books About Me Contact Me ThoughtWorks

Inversion of Control Containers and the Dependency Injection pattern

Martin Fowler

In the Java community there's been a rush of lightweight containers that help to assemble components from different projects into a cohesive application. Underlying these containers is a common pattern to how they perform the wiring, a concept they refer under the very generic name of "Inversion of Control". In this article I dig into how this pattern works, under the more specific name of "Dependency Injection", and contrast it with the Service Locator alternative. The choice between them is less important than the principle of separating configuration from use.

Last significant update: 23 Jan 04

| Chinese | Portuguese | French | Italian |

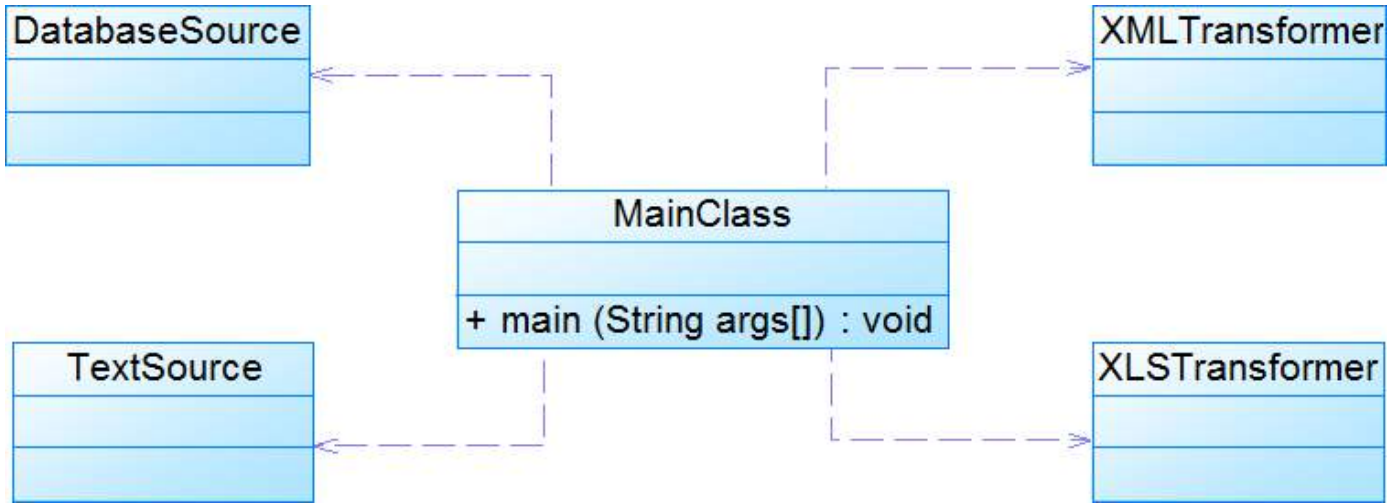
Contents

依赖倒转原则

- 依赖倒转原则分析
  - 依赖注入
    - 构造注入（Constructor Injection）：通过构造函数注入实例变量
    - 设值注入（Setter Injection）：通过Setter方法注入实例变量
    - 接口注入（Interface Injection）：通过接口方法注入实例变量

依赖倒转原则

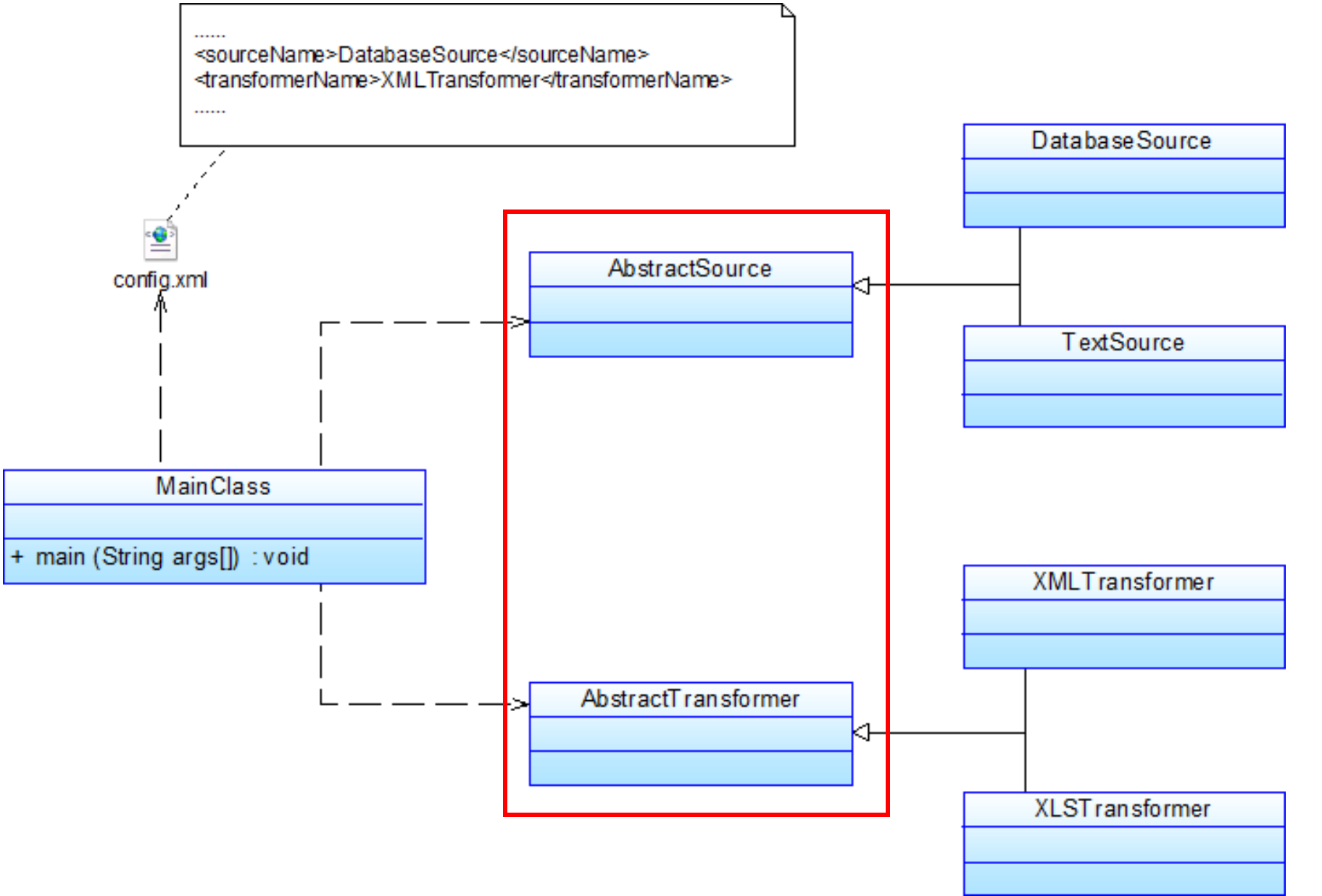
- 依赖倒转原则实例
  - 实例说明
    - 某系统提供一个数据转换模块，可以将来自不同数据源的数据转换成多种格式，如可以转换来自数据库的数据（DatabaseSource）、也可以转换来自文本文件的数据（TextSource），转换后的格式可以是XML文件（XMLTransformer）、也可以是XLS文件（XLSTransformer）等



依赖倒转原则

- 依赖倒转原则实例
  - 实例说明
    - 由于需求的变化，该系统可能需要增加新的数据源或者新的文件格式，每增加一个新的类型的数据源或者新的类型的文件格式，客户类MainClass都需要修改源代码，以便使用新的类，但违背了开闭原则。现使用依赖倒转原则对其进行重构

依赖倒转原则


.....
<sourceName>DatabaseSource</sourceName>
<transformerName>XMLTransformer</transformerName>
.....
config.xml
DatabaseSource
AbstractSource
TextSource
MainClass
+ main (String args[]) : void
XMLTransformer
AbstractTransformer
XLSTransformer

接口隔离原则

- 接口隔离原则定义
  - 接口隔离原则（Interface Segregation Principle, ISP）
    - 客户端不应该依赖那些它不需要的接口
    - 注意：在该定义中的接口指的是所定义的方法
    - 一旦一个接口太大，则需要将它分割成一些更细小的接口，使用该接口的客户端仅需知道与之相关的方法即可

接口隔离原则

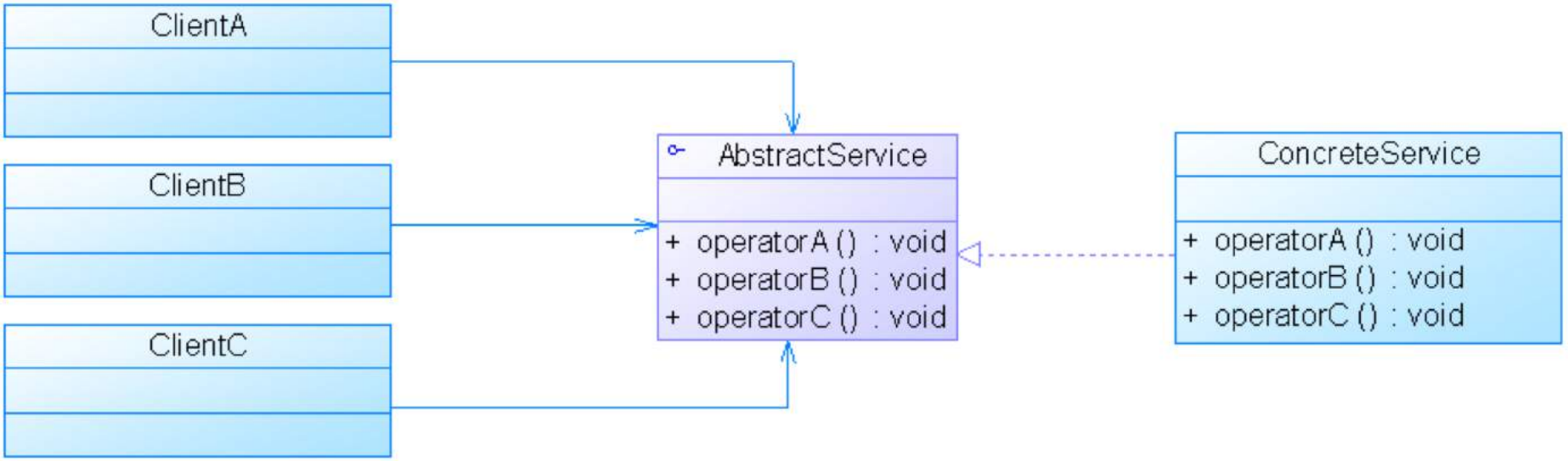
- 接口隔离原则分析
  - 接口隔离原则是指使用多个专门的接口，而不使用单一的总接口。每一个接口应该承担一种相对独立的角色，不多不少，不干不该干的事，该干的事都要干
    - 一个接口只代表一个角色，每个角色都有它特定的接口，此时这个原则可以叫做“角色隔离原则”
    - 接口仅仅提供客户端需要的行为，即所需的方法，客户端不需要的行为则隐藏起来，应当为客户端提供尽可能小的单独的接口，而不要提供大的总接口

接口隔离原则

- 接口隔离原则分析
  - 使用接口隔离原则拆分接口时，首先必须满足单一职责原则，将一组相关的操作定义在一个接口中，且在满足高内聚的前提下，接口中的方法越少越好
  - 可以在进行系统设计时采用定制服务的方式，即为不同的客户端提供宽窄不同接口，只提供用户需要的行为，而隐藏用户不需要的行为

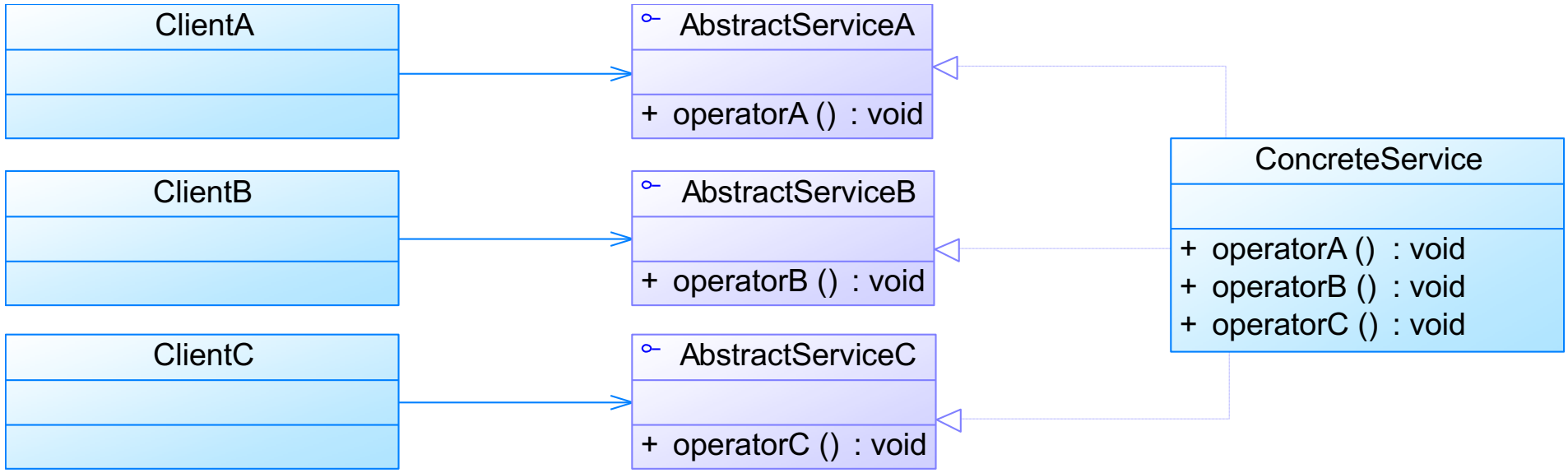
接口隔离原则

- 接口隔离原则实例
  - 实例说明
    - 下图展示了一个拥有多个客户类的系统，在系统中定义了一个巨大的接口（胖接口）AbstractService来服务所有的客户类
    - 使用接口隔离原则对其进行重构



接口隔离原则

- 接口隔离原则实例
  - 实例解析



合成复用原则

- 合成复用原则定义
  - 合成复用原则（Composite Reuse Principle, CRP）
    - 尽量使用对象组合，而不是继承来达到复用的目的

Favor composition of objects over inheritance as a reuse mechanism

合成复用原则

- 合成复用原则分析
  - 可以通过两种基本方法在不同的环境中复用已有的设计和实现，即通过组合/聚合关系或通过继承
    - 继承复用：实现简单，易于扩展。破坏系统的封装性；从基类继承而来的实现是静态的，不可能在运行时发生改变，没有足够的灵活性；只能在有限的环境中使用。（“白箱”复用）
    - 组合/聚合复用：耦合度相对较低，选择性地调用成员对象的操作；可以在运行时动态进行。（“黑箱”复用）

合成复用原则

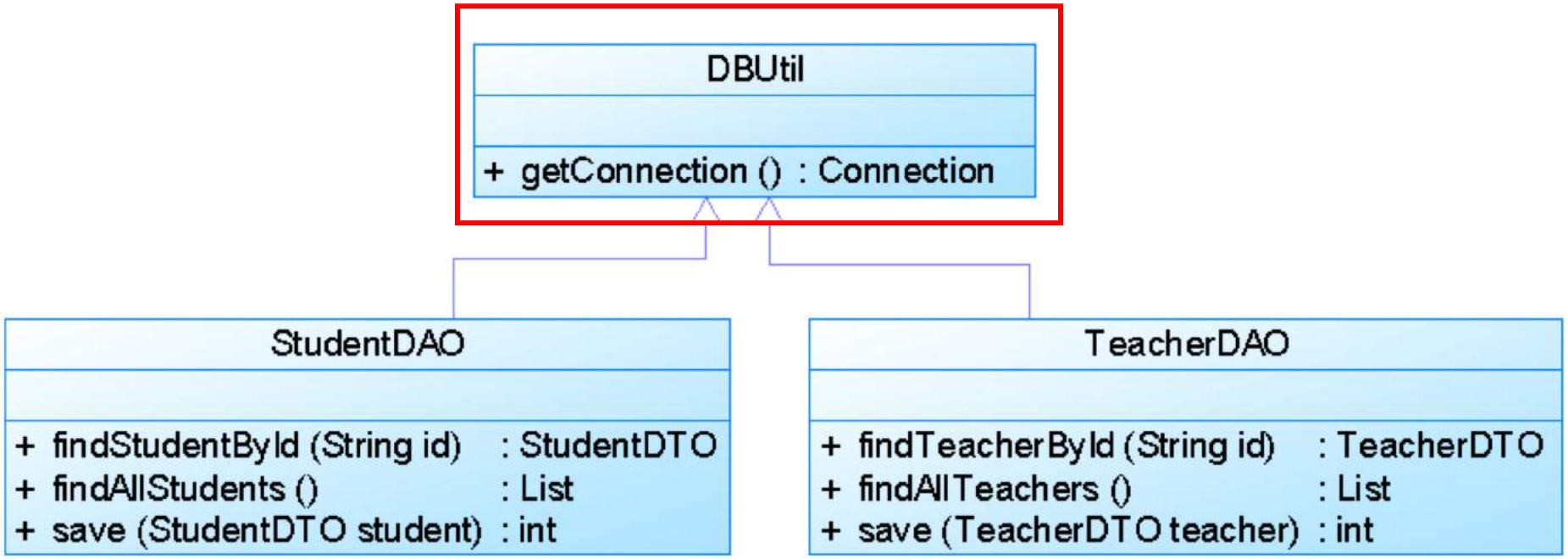
- 合成复用原则分析
  - 组合/聚合可以使系统更加灵活，类与类之间的耦合度降低，一个类的变化对其他类造成的影响相对较少
  - 因此一般首选使用组合/聚合来实现复用；其次才考虑继承，在使用继承时，需要严格遵循里氏代换原则
  - 有效使用继承会有助于对问题的理解，降低复杂度，而滥用继承反而会增加系统构建和维护的难度以及系统的复杂度，因此需要慎重使用继承复用

合成复用原则

- 合成复用原则分析
  - 合成复用原则就是指在一个新的对象里通过关联关系（包括组合关系和聚合关系）来使用一些已有的对象，使之成为新对象的一部分
  - 新对象通过委派调用已有对象的方法达到复用其已有功能的目的
  - 尽量使用组合/聚合关系，少用继承

合成复用原则

- 合成复用原则实例
  - 实例说明
    - 某教学管理系统部分数据库访问类设计如图所示

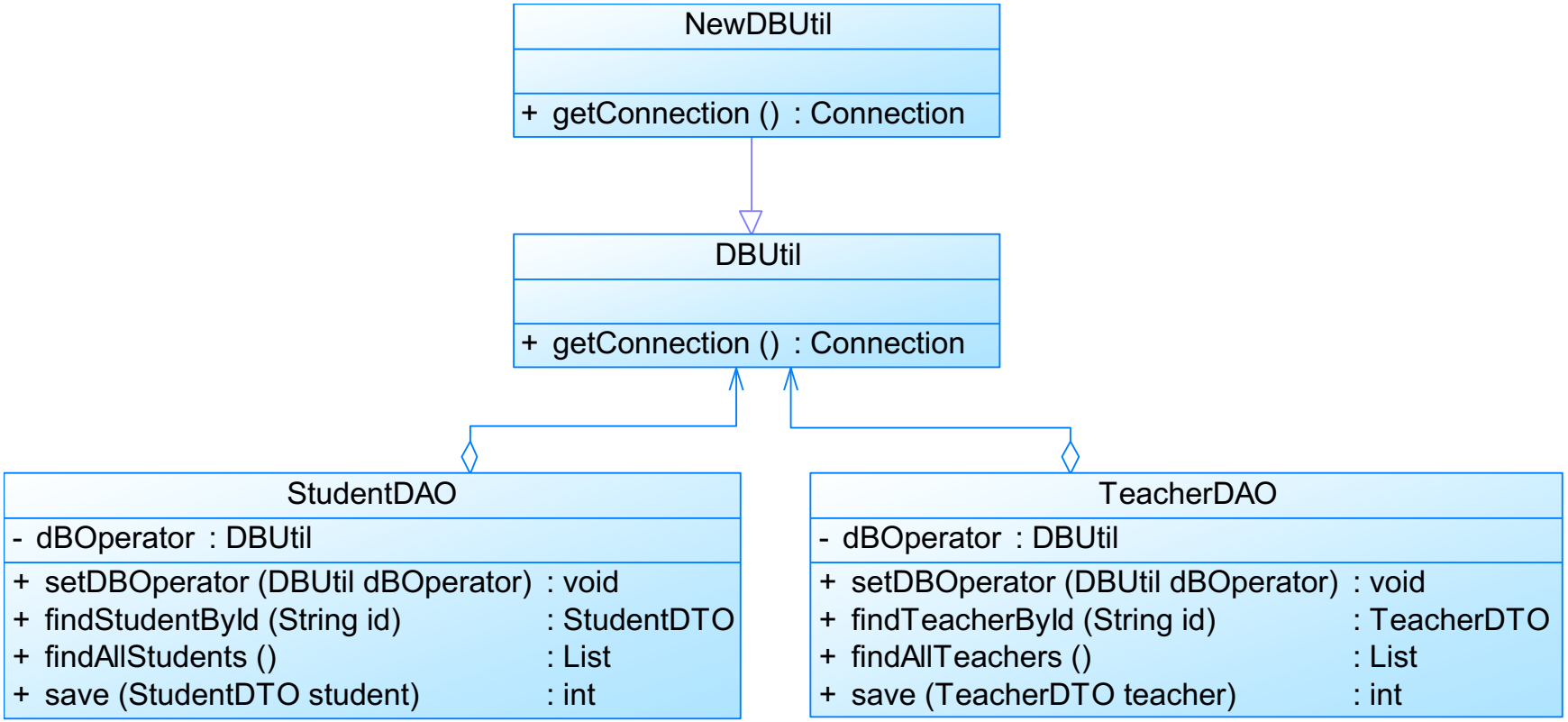


合成复用原则

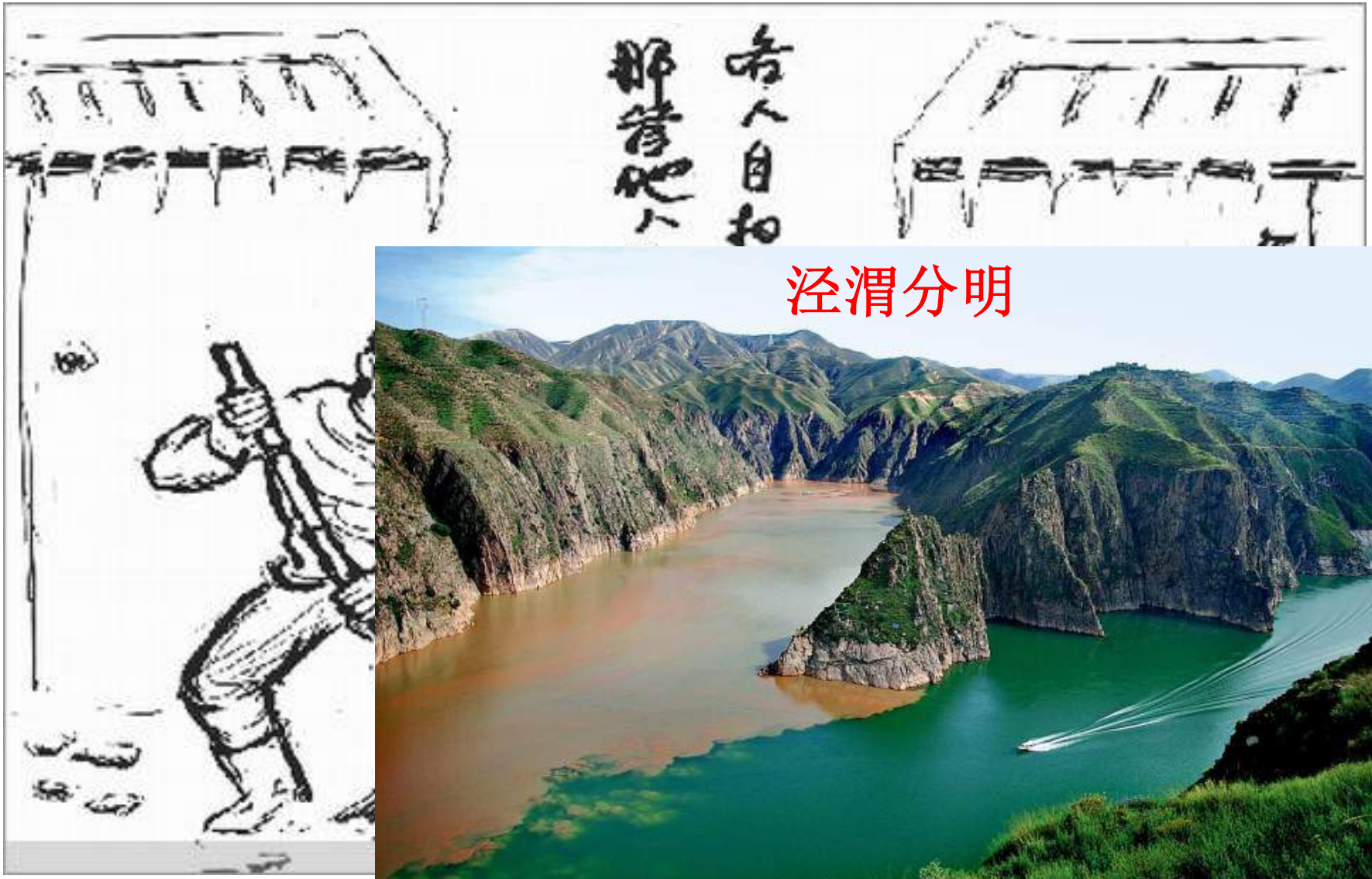
- 合成复用原则实例
  - 实例说明
    - 如果需要更换数据库连接方式，如原来采用JDBC连接数据库，现在采用数据库连接池连接，则需要修改DBUtil类源代码
    - 如果StudentDAO采用JDBC连接，但是TeacherDAO采用连接池连接，则需要增加一个新的DBUtil类，并修改StudentDAO或TeacherDAO的源代码，使之继承新的数据库连接类，这将违背开闭原则，系统扩展性较差
    - 现使用合成复用原则对其进行重构

合成复用原则

- 合成复用原则实例
  - 实例解析



迪米特法则



迪米特法则

- 迪米特法则来源
  - 1987年秋美国东北大学（Northeastern U
    个名为Demeter的研究项目
  - 简单地说，迪米特法则就是指一个软件实
    少的与其他实体发生相互作用
  - 当一个模块修改时，就会尽量少的影响其
    展会相对容易
  - 这是对软件实体之间通信的限制，它要求
    之间通信的宽度和深度



Demeter/ Ceres

希腊神话中司掌农业、谷物和母爱的地母之神

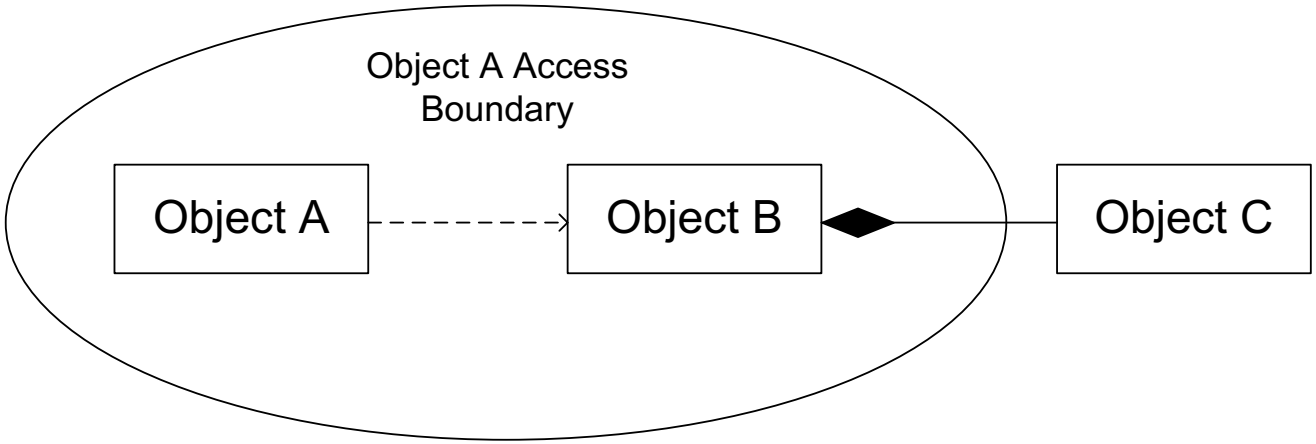
迪米特法则

- 迪米特法则分析
  - 对于一个对象，其朋友包括以下几类：
    - 当前对象本身（this）
    - 以参数形式传入到当前对象方法中的对象
    - 当前对象的成员对象或集合
    - 当前对象所创建的对象
  - 任何一个对象，如果满足上面的条件之一，就是当前对象的“朋友”，否则就是“陌生人”



迪米特法则

- 迪米特法则分析
  - 狭义迪米特法则：如果两个类之间不必彼此直接通信，那么这两个类就不应当发生直接的相互作用，如果其中的一个类需要调用另一个类的某一个方法的话，可以通过第三者转发这个调用
  - 可以降低类之间的耦合，但是会在系统中增加大量的小方法并散落在系统的各个角落



迪米特法则

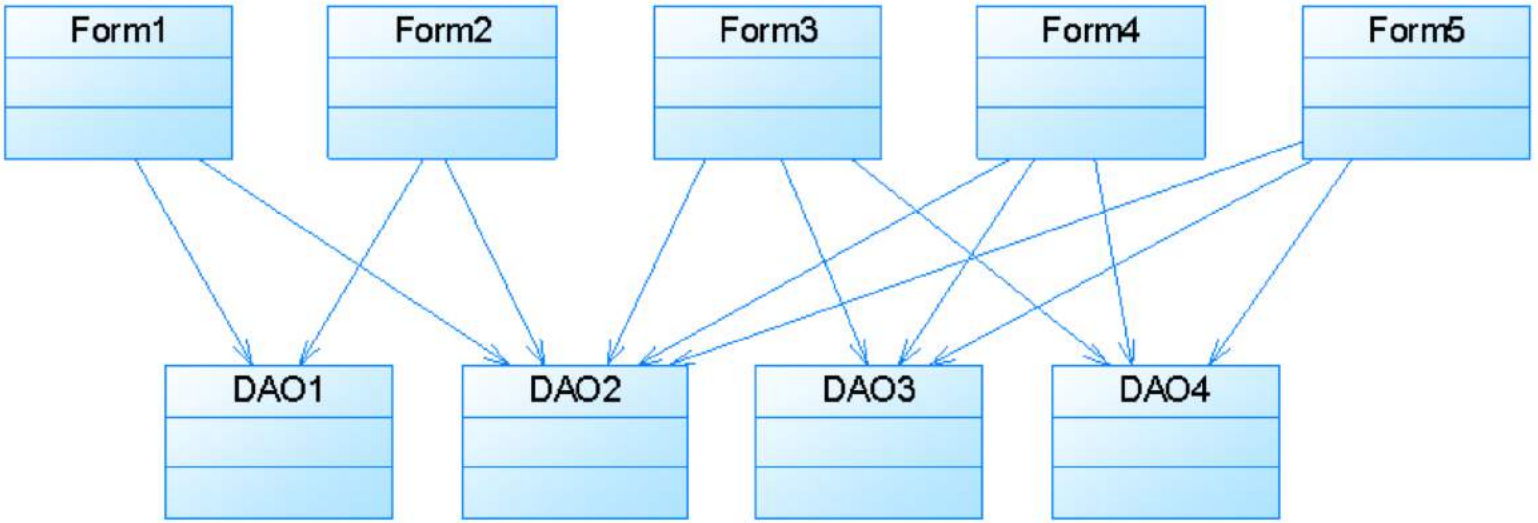
- 迪米特法则分析
  - 广义迪米特法则：指对对象之间的信息流量、流向以及信息的影响的控制，主要是对信息隐藏的控制
  - 信息的隐藏可以使各个子系统之间脱耦，从而允许它们独立地被开发、优化、使用和修改
  - 由于每一个模块都不依赖于其他模块而存在，因此每一个模块都可以独立地在其他的地方使用
  - 一个系统的规模越大，信息的隐藏就越重要，而信息隐藏的重要性也就越明显

迪米特法则

- 迪米特法则分析
  - 迪米特法则的主要用途在于控制信息过载：
    - 在类的划分上，应当尽量创建松耦合的类，类之间的耦合度越低，就越有利于复用
    - 在类的结构设计上，每一个类都应当尽量降低其成员变量和成员函数的访问权限
    - 在类的设计上，只要有可能，一个类型应当设计成不变类
    - 在类的引用上，一个对象对其他对象的引用应当降到最低

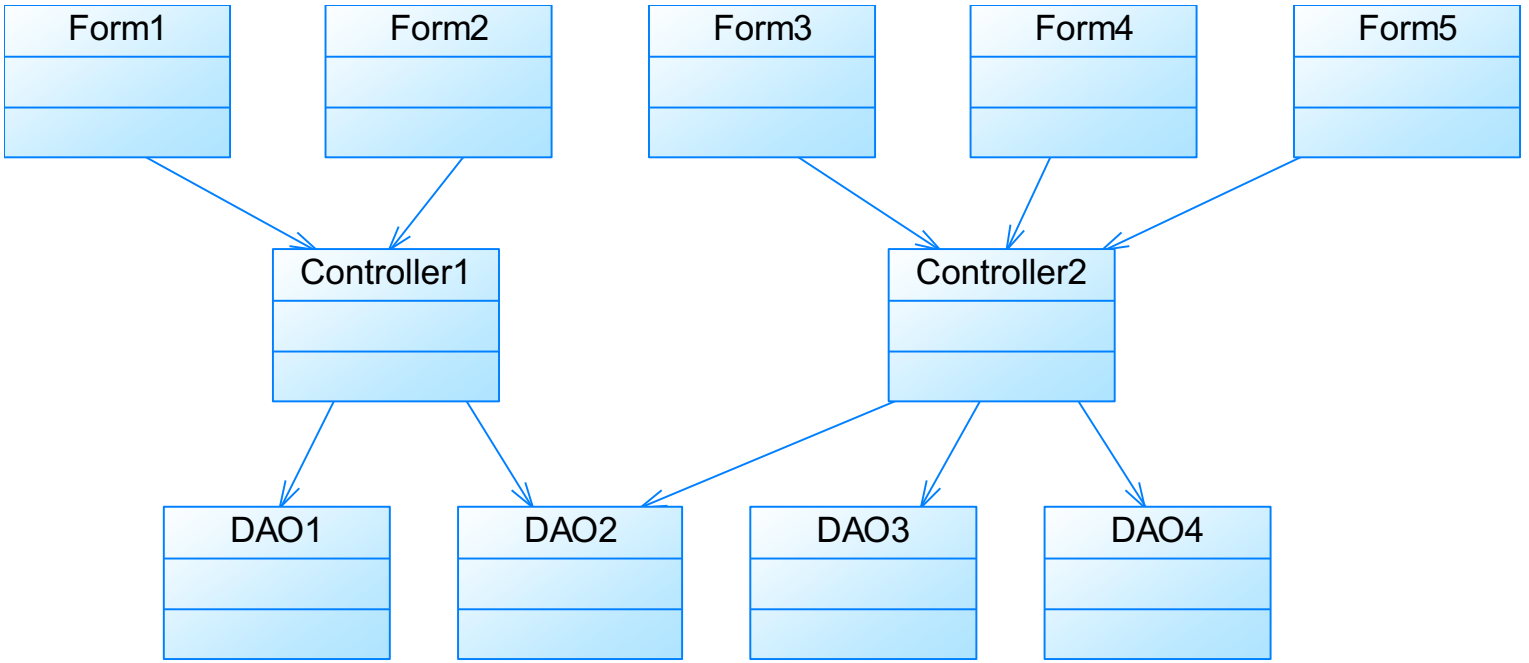
迪米特法则

- 迪米特法则实例
  - 实例说明
    - 某系统界面类（如Form1、Form2等类）与数据访问类（如DAO1、DAO2等类）之间的调用关系较为复杂
    - 运用迪米特法则对其进行重构



迪米特法则

- 迪米特法则实例
  - 实例解析



本章小结

- 对于面向对象的软件系统设计来说，在支持可维护性的同时，需要提高系统的可复用性
- 软件的复用可以提高软件的开发效率，提高软件质量，节约开发成本，恰当的复用还可以改善系统的可维护性

本章小结

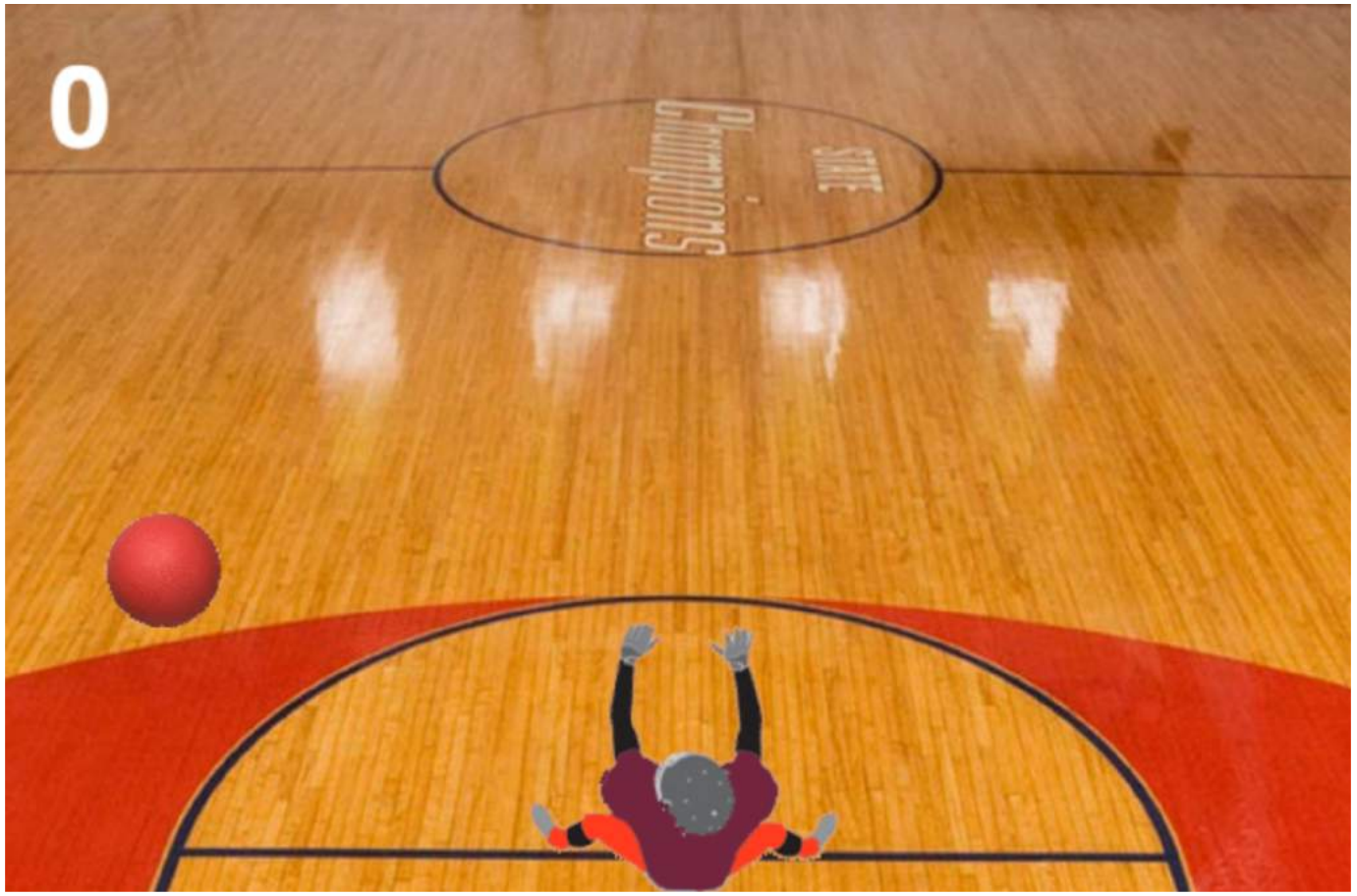
- 单一职责原则要求在软件系统中，一个类只负责一个功能领域中的相应职责
- 开闭原则要求一个软件实体应当对扩展开放，对修改关闭，即在不修改源代码的基础上扩展一个系统的行为
- 里氏代换原则可以通俗表述为在软件中如果能够使用基类对象，那么一定能够使用其子类对象
- 依赖倒转原则要求抽象不应该依赖于细节，细节应该依赖于抽象；要针对接口编程，不要针对实现编程
- 接口隔离原则要求客户端不应该依赖那些不需要的接口，即将一些大的接口细化成一些小的接口供客户端使用
- 合成复用原则要求复用时尽量使用对象组合，不使用继承
- 迪米特法则要求一个软件实体应尽可能少的与其他实体发生相互作用

第二部分

面向对象案例分析

面向对象案例分析

- Dodgeball
  - http://davidk.net/game/




HTML5 Games
with CreateJS
TWITTER: @DAVID_KELLEHER
DAVIDK.NET/GAME

CreateJS Features

HTML5 Canvas 2D Context Framework

Pros

- Bitmaps
- Vectors
- Animation
- Sprite Sheets
- Containers
- Flash Style Moveiclips
- Inputs/Events
- Text
- Sounds
- Preloader

Cons

- No Level Editor
- No Physics
- No Particles
- No Virtual Camera
- Beta WebGL 2D Support
- No 3D Support
- No UI Controls
- No Networking

CreateJS Uses

Pros

- Leverage Adobe Flash skills and current support for Adobe Animate
- Create *simple* Flash-style games, banner ads, data viz, generative art
- Rapidly prototype 2D games
- Integrate with Angular, React, D3

Cons

- Not a high performance 2D library
- Not a true game development framework
- Not production ready – current version 0.8.2

CreateJS Performance

Pros

- Canvas hardware acceleration
- Caching of drawn objects - transform regions with complex vector graphics or many display objects

Cons

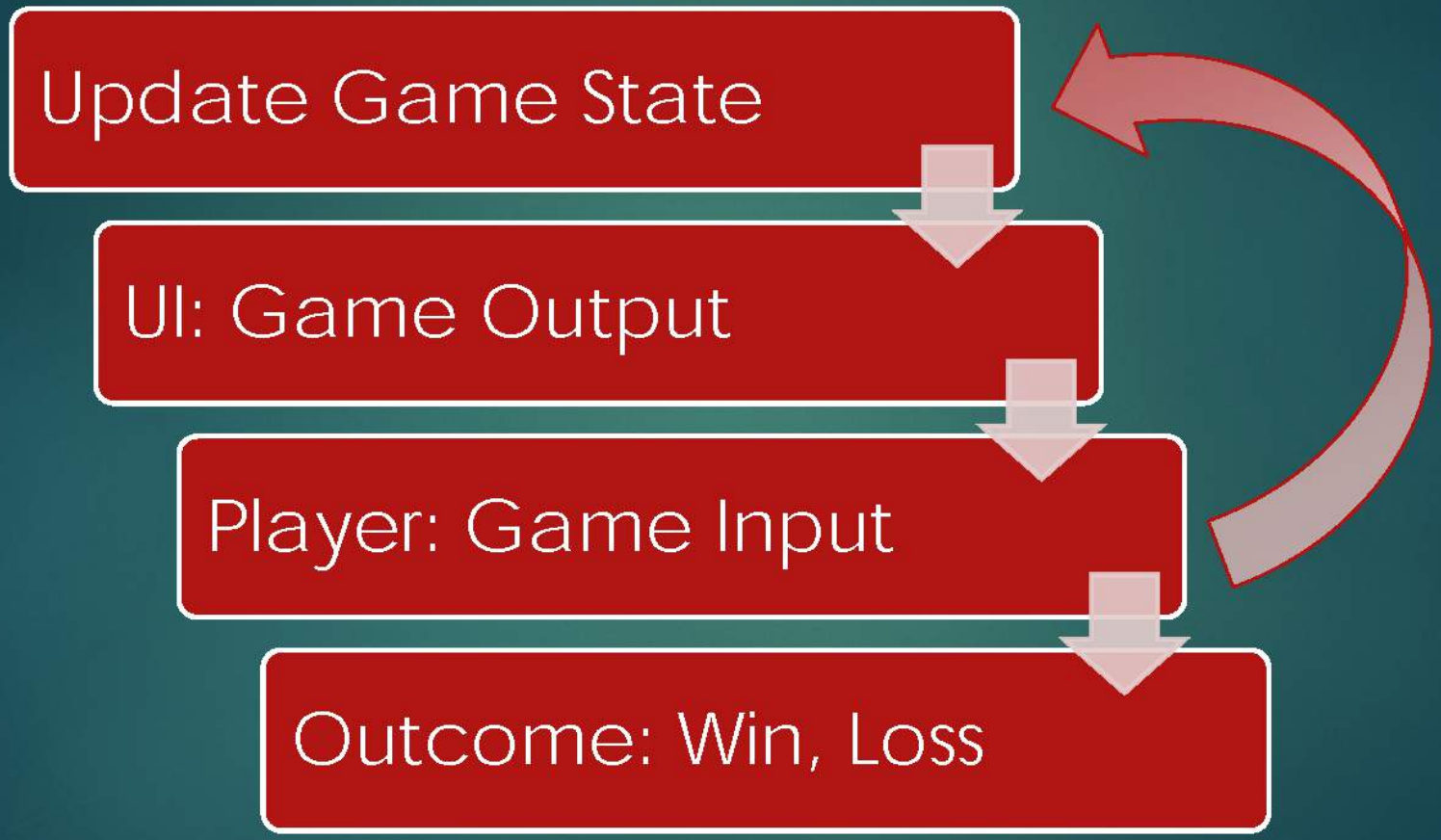
- No redraw regions – calculation time too expensive in JavaScript for arbitrary content
- Slow, very basic hit test

Architecture

- CreateJS – extends the HTML5 <canvas>
  - EaselJS
  - TweenJS
  - SoundJS
  - PreloadJS

Game Theory

Game Loop


Update Game State
UI: Game Output
Player: Game Input
Outcome: Win, Loss

Game Mechanics

- Strategy
- Dexterity
- Luck

Game Input and Events

User Input

- Mouse Move
- Click, Mouseup, Mousedown
- Drag and Drop, Swipe
- Key Input

Game Event

- Ticker
- Collision (Hit Test)
- Custom

Update Game State

Game Objects

- Add/Remove Objects
- Change Object Properties
- Tween Animations
- Update Score
- Test Win/Loss Condition

Dodgeball Inputs & Events

- Player's avatar tracks horizontal movement of mouse
- Balls can be clicked allowing players to catch them
- Game events: Detect collisions each frame between player and balls

Update Game State

- Add balls and remove when off court
- Move balls (motion tween)
- Change ball state when it is clicked, caught, or hits player
- Add point when ball is caught
- Change player state and end game when player hit by ball

Coding Strategy

- Avoid creating a massively complex controller with all procedural coding in the game loop.

Object Oriented

- Blinky speeds up as you eat dots



- Pinky tends to move counterclockwise



- Inky makes unpredictable turns



- Clyde doesn't always chase pac man



Dave's Dodgeball Game

Canvas Element

Canvas Element

```
// HTML5 CANVAS ELEMENT
var canvas = document.createElement("canvas");
canvas.width = 600;
canvas.height = 400;

// --- Add canvas element to DOM
document.body.appendChild(canvas);
```

CreateJS Stage

CreateJS Stage

```
// CreateJS STAGE   // Uses 2D Canvas Context
var stage = new createjs.Stage(canvas);
stage.mouseMoveOutside = true;
```

CreateJS Ticker

EaselJS – Ticker Class

- Has a static interface
- Properties
  - framerate: frames per second
  - interval: milliseconds between ticks
  - maxDelta: max time between ticks
  - paused: pause the ticker
- Methods
  - reset(): stop ticker
  - init(): restart ticker
  - getMeasuredFPS(): returns actual ticks per second
  - getTicks(): total number of ticks broadcast
  - getMeasuredTickTime(): avg tick execution time in ms
  - getTime(): milliseconds since ticker started

CreateJS Ticker

```
// CreateJS TICKER  // Attaches Game Loop to Stage
createjs.Ticker.on("tick", stage);
createjs.Ticker.framerate = 60;
```

Gym Background Object

EaselJS – Bitmap Class

- Pass a URI to an image file, or a reference to an HTML Image, HTML Video, or HTML Canvas element to the constructor.
- Don't cache a single bitmap, unless you want to use a blur, alpha, or color filter, then you have to.
- Properties:
  - image: swap a new image into the object
  - mask: shape image defining a clipping path
  - mouseEnabled: enable/disable mouse event
- Transform: x, y, scaleX, ScaleY, rotate, regX, regY

Gym Object

```
// GYM BACKGROUND OBJECT
var gym = new createjs.Bitmap("image/gym.jpg");

// --- Add gym to stage
stage.addChild(gym);
```

Score Object

EaselJS – Text Class

- Pass the text, optional CSS font, and optional CSS color to the constructor.

- Properties:
  text: change the text
  color: change the color
  font: change the font
  outline: add stroke to the text
  shadow: add drop shadow to the text

Score Object

```
// SCORE OBJECT
var score = new createjs.Text(0, 'bold 50px sans-serif', '#FFF');
score.x = 20;
score.y = 20;

// --- custom properties
score.value = 0;

// --- Add player to stage
stage.addChild(score);
```

Player Object

Player Graphics

```
// PLAYER GRAPHICS
var playerAlive = new Image();
playerAlive.src = "image/player.png";

var playerHit = new Image();
playerHit.src = "image/player-dead.png";
```

Player Object

```
// PLAYER OBJECT
var player = new createjs.Bitmap();
player.image = playerAlive;
player.x = 232;
player.y = 275;
```

CreateJS Listeners

- addEventListener
  pass event string and handler function
  addEventListener("click", handleClick);

- removeEventListener
  pass exact function used when added
  removeEventListener("click", handleClick);

CreateJS Listeners

- on

  shortcut method for addEventListener
  also specify execution scope, only run once, etc…
  var listener = myBtn.on("click", handleClick, null, false);

- off

  pass returned wrapper function
  myBtn.on("click", listener);

Mouse Handler: Player

```
// --- custom method for moving player when mouse moved
player.move = function (mouseEvent) {
      player.x = mouseEvent.stageX - 68;
}

// --- handle mouse movements
player.moveListener = stage.on("stagemousemove", player.move);
```

Player: Add to Stage

```
// --- Add player to stage
stage.addChild(player);
```

Ball Class

Ball Graphics

```
// BALL GRAPHICS
var ballTossed = new Image();
ballTossed.src = "image/ball.png";

var ballCatchable = new Image();
ballCatchable.src = "image/ball-catch.png";

var ballCaught = new Image();
ballCaught.src = "image/star.gif";
```

Multiple Game Objects

- Option #1 – CreateJS clone method
- Option #2 – JavaScript class

Ball "Class"

```
// BALL CLASS
var Ball = function() {
    var ball = new createjs.Bitmap();
    ball.image = ballTossed;
    ball.x = Math.floor((Math.random() * 600) + 1);
    ball.scaleX = 0.25;
    ball.scaleY = 0.25;
    ball.regX = 50;
    ball.regY = 50;
    ...
```

Ball: Custom Properties

```
// BALL CLASS
var Ball = function() {
    ...
    // --- custom properties
    ball.state = "Tossed";
    ball.moveToX = Math.floor((Math.random() * 600) + 1);
    ball.moveTime = Math.floor((Math.random() * 2200) + 3200);
    ...
```

TweenJS

- Implements motion tweens
- Can be chained to create animation sequences
- Can be stacked, but should target different properties with each tween
- Dispatches a change event when a property is changed
- Supports many ease types http://www.createjs.com/demos/tweenjs/tween_sparktable

TweenJS example

```
createjs.Tween.get(target)
    .wait(500)
    .to({alpha:0, visible:false}, 1000)
    .call(handleComplete);

function handleComplete() {
    //Tween complete
}
```

Ball: Animation

```
// BALL CLASS
var Ball = function() {
    ...
    // --- move the ball using TweenJS
    createjs.Tween.get(ball)
        .to( {  scaleX: 0.75,
                scaleY: 0.75,
                x: ball.moveToX,
                y: 500,
                rotation: 1440 },
            ball.moveTime,
            createjs.Ease.getPowOut(3)
        );
    ...
```

Click Handler: Ball

```
// BALL CLASS
var Ball = function() {
    ...
    // --- custom method for handling mouse clicks on ball
    ball.touched = function () {
        ball.state="Catchable";
        ball.image = ballCatchable
    }

    // --- handle mouse clicks
    ball.clickListener = ball.on("mousedown", ball.touched);
    ...
```

Click Handler: Ball

```
// BALL CLASS
var Ball = function() {
    ...
    // --- Provide access to the ball object
    return ball;
}
```

Ball Object Factory

Ball Objects

```
// BALL OBJECTS
// --- Call addBall method EVERY FRAME
var addBall = createjs.Ticker.on("tick", function addBall () {
    randomNumber = Math.floor((Math.random() * 60) + 1);
    if (randomNumber === 1) {
        stage.addChild(new Ball());
    }
});
```

Collisions

Custom Event

- Event Class
  Create an event string object

- EventDispatcher Class
  Dispatch event to listeners

Custom Event example

```
var event = new createjs.Event("progress");
this.dispatchEvent(event);
```

Collision: Player Catch Ball

```
// BALL CLASS
var Ball = function() {
    ...
    // --- custom method called when player catches ball
    ball.playerCatchesBall = function () {
        // Change the ball to the caught state
        ball.state = "Caught";
        // Change the ball image to the spinning star
        ball.image = ballCaught;
        // Make the star spin away by moving the registration point
        ball.regX = 130;
        ball.regY = 130;
        // Dispatch event to score a point
        var caughtballEvent = new createjs.Event("caughtball");
        stage.dispatchEvent(caughtballEvent);
    }
```

Collision: Ball Hits Player

```
// BALL CLASS
var Ball = function() {
    ...
    // --- custom method called when ball hits player
    ball.ballHitsPlayer = function() {
        // Displatch event to remove ball click handlers, stop player
        var hitbyballEvent = new createjs.Event("hitbyball");
        stage.dispatchEvent(hitbyballEvent);
    }

    // --- handle ball hitting player, remove handler for clicking ball
    stage.on("hitbyball", function () {
        ball.off("mousedown", ball.clickListener);
    });
    ...
```

Collision: Hit Test

```
// BALL CLASS
var Ball = function() {
    ...
    // --- custom method for performing hit tests
    ball.hitTest = function () {
        if (this.y > 500) {
            // The ball passed by the player and is off the stage
            stage.removeChild(this);
        } else {
            // Hit Test
            ...
        }
    }
    ...
```

CreateJS Hit Tests

- hitTest() method
  - test if object intersects a point
  - must convert point to local coordinates
  - not very useful, not great performance
- Shape to Shape collisions not supported
  - can code your own with bounding boxes
  - getBounds() method returns rectangle
- Pythagorean Theorem
  - simple hit test approximation
  - gets distance between two points

Collision: Hit Test (cont.)

```
...
// Hit Test
var xDist = this.x - player.x - 70;
var yDist = this.y - player.y - 30;
var distance = Math.sqrt(xDist * xDist + yDist * yDist);
// Using pythagorus
if (distance < 50) {
        if (ball.state === "Catchable") {
                ball.playerCatchesBall();
        } else if (ball.state === "Tossed")  {
                ball.ballHitsPlayer();
        }
}
...
```

Add Hit Test to Game Loop

```
// BALL CLASS
var Ball = function() {
    ...
    // Call hitTest method EVERY FRAME
    ball.on("tick", ball.hitTest);
    ...
```

Collision Handler: Score

```
// SCORE  OBJECT
...
// --- custom method for scoring a point when ball caught
score.catch = function (mouseEvent) {
        score.value++;
        score.text = score.value;
}

// --- handle catching the ball
score.catchListener = stage.on("caughtball", score.catch);
```

Collision Handler: Player

```
// PLAYER  OBJECT
...
// --- custom method for ending game when player hit by ball
player.hit = function () {
      player.image = playerHit;
      stage.off("stagemousemove", player.moveListener);
}

// --- handle getting hit by a ball
player.hitListener = stage.on("hitbyball", player.hit);
```

Collision Handler: Balls

```
// BALL  OBJECT FACTORY
...
// --- handle getting hit by a ball, stop the bombardment
stage.on("hitbyball", function () {
      createjs.Ticker.off("tick", addBall);
});
```

Additional Links

- CreateJS

http://www.createjs.com/

- Pong Tutorial

http://code.tutsplus.com/tutorials/learn-createjs-by-building-an-html5-pong-game--active-11845

- Atari Arcade

https://www.atari.com/arcade/developers
http://www.htmlgoodies.com/html5/client/building-atari-with-createjs.html

- Atari Arcade SDK (extends CreateJS)

https://github.com/AtariAdmin/AtariArcadeSDK

面向对象案例分析

- 可编程魔杖

END





Thanks!